

that it accepts a fourth parameter, the affinity field. It indicates the thread on which the computation runs. It can be an integer that is internally represented as *integer % THREADS* or it can be an address corresponding to a thread. The program can be compiled and tested with:

```
# gupc upc_vector_addition.c -o upc_vector_addition
# ./upc_vector_addition -n 5
6 6 6 6 6
```

The same example can also be implemented using shared pointers:

```
#include <upc_relaxed.h>
#include <stdio.h>
shared int a[THREADS];
shared int b[THREADS];
shared int vsum[THREADS];
int
main()
{
    int i;
    shared int *p1, *p2;
    p1 = a;
    p2 = b;
    /* Initialization */
    for (i=0; i<THREADS; i++) {
        *(p1 + i) = i + 1; /* a[] = {1, 2, 3, 4, 5}; */
        *(p2 + i) = THREADS - i; /* b[] = {5, 4, 3, 2, 1}; */
    }
    /* Computation */
    upc_forall (i=0; i<THREADS; i++, p1++, p2++; i)
        vsum[i] = *p1 + *p2;
    upc_barrier;
    if (MYTHREAD == 0)
        for (i = 0; i < THREADS; i++)
            printf("%d ", vsum[i]);
    return 0;
}

# gupc pointer_vector_addition.c -o pointer_vector_addition
# ./pointer_vector_addition -n 5
6 6 6 6 6
```

Memory can also be allocated dynamically. The *upc_all_alloc* function will allocate collective global memory that is shared among threads. A collective function will be invoked by every thread. The *upc_global_alloc* function will allocate non-collective global memory, which will be different for all threads in the shared address space. The *upc_all_alloc* function will allocate local memory for a thread. Their respective declarations are as follows:

```
shared void *upc_all_alloc (size_t nblocks, size_t nbytes);
shared void *upc_global_alloc (size_t nblocks, size_t
```

```
nbytes);
shared void *upc_alloc (size_t nbytes);
```

To protect access to shared data, you can use the following synchronisation locks:

```
void upc_lock (upc_lock_t *l)
int upc_lock_attempt (upc_lock_t *l)
void upc_unlock (upc_lock_t *l)
```

There are two types of barriers for synchronising code. The *upc_barrier* construct is for blocking. The non-blocking barrier uses *upc_notify* (non-blocking) and *upc_wait* (blocking) constructs. For example:

```
#include <upc_relaxed.h>
#include <stdio.h>
int
main()
{
    int i;
    for (i=0; i<THREADS; i++) {
        upc_notify;
        if (i == MYTHREAD)
            printf("Thread: %d\n", MYTHREAD);
        upc_wait;
    }
    return 0;
}
```

The corresponding output is shown below:

```
# gupc count.c -o count
# ./count -n 5
Thread: 0
Thread: 1
Thread: 2
Thread: 3
Thread: 4
```

You can refer to the GUPC user guide for more information. 

References

- [1] GNU Unified Parallel C. <http://www.gnu.org/gccupc/>
- [2] Unified Parallel C extension. <https://upc-lang.org/>
- [3] UPC Specification. <http://upc-specification.googlecode.com/>
- [4] GUPC User Guide. <http://www.gnu.org/documents/gupc-user-doc.html>

By: Shakthi Kannan

The author is a free software enthusiast and blogs at shakthimaan.com